

# CSSD 2211 DOCUMENTATION

## Legends of Sword and Wand: Technical Documentation

### 1. Project Overview

Legends of Sword and Wand is a browser-based fantasy RPG built on a microservices architecture. The system consists of four independently deployable services communicating over HTTP REST APIs:

- **Party Service:** Handles hero creation, stats, leveling, and ability resolution (Port 5001).  
**PvE Service:** Manages campaigns, map navigation, inn interactions, and scoring (Port 5002).  
+2
- **Battle Service:** Manages PvP invitations, battle simulation, and league standings (Port 5003).
- **Frontend:** A React/Vite web UI that consumes all three backend services (Port 5173).

## 2. Primary API Endpoints

### 2.1 Party Service (Port 5001)

- POST /hero/create: Create a new hero (requires username, hero\_name, initial\_class).
- GET /hero/<hero\_id>: Dynamically compute stats and abilities.
- POST /hero/<hero\_id>/level\_up: Increment a specific class level.

### 2.2 PvE Service (Port 5002)

- POST /auth/login: Login or auto-register.
- POST /campaign/start: Begin a 30-room journey.
- POST /campaign/{id}/next-room: Advance the player.
- POST /campaign/{id}/battle-result: Report outcome and distribute rewards.
- POST /campaign/{id}/score: Calculate final score for the Hall of Fame.

### 2.3 Battle / PvP Service (Port 5003)

- POST /pvp/invite: Send battle invitation.
- POST /pvp/invite/<id>/respond: Accept or decline an invite.
- POST /pvp/battle/start: Execute a simulated PvP battle.
- GET /pvp/league: Retrieve standings sorted by wins.

# Communication Between Systems

## 1. The Core Protocol: HTTP REST

All inter-service communication happens exclusively over HTTP REST APIs. The system completely avoids shared memory or message queues. Each service also maintains its own independent database (MongoDB, PostgreSQL, or SQLite) and data is only transferred via these API calls.

## 2. The Frontend Acts as the API Gateway

Instead of having a separate routing container (like Nginx), the Vite development server embedded in the Frontend container acts as the API gateway. It handles routing in two distinct ways:

Proxy Routing (PvE and Party Services): The frontend uses a Vite proxy block to intercept requests based on their URL prefix. If the frontend makes a call to `/pve/*`, Vite rewrites the path and proxies it to the PvE service on port 8000. Calls to `/party/*` are similarly proxied to the Party service on port 5001.

Direct Routing (Battle Service): The frontend communicates directly with the Battle Service on port 8001, completely bypassing the proxy. This is intentionally designed this way because turn-based battle actions are frequent and latency-sensitive.

## 3. The "Frontend Orchestrator" Pattern

Instead of the backend microservices constantly talking directly to each other during gameplay, the Frontend acts as the middleman orchestrating the game loop.

Here is how that works in practice during a PvE campaign:

The frontend asks the PvE service for the next room and gets the enemy party data.

The frontend takes that enemy data and its own hero data, and directly calls the Battle Service to start the fight.

The frontend and Battle Service go back and forth for every turn.

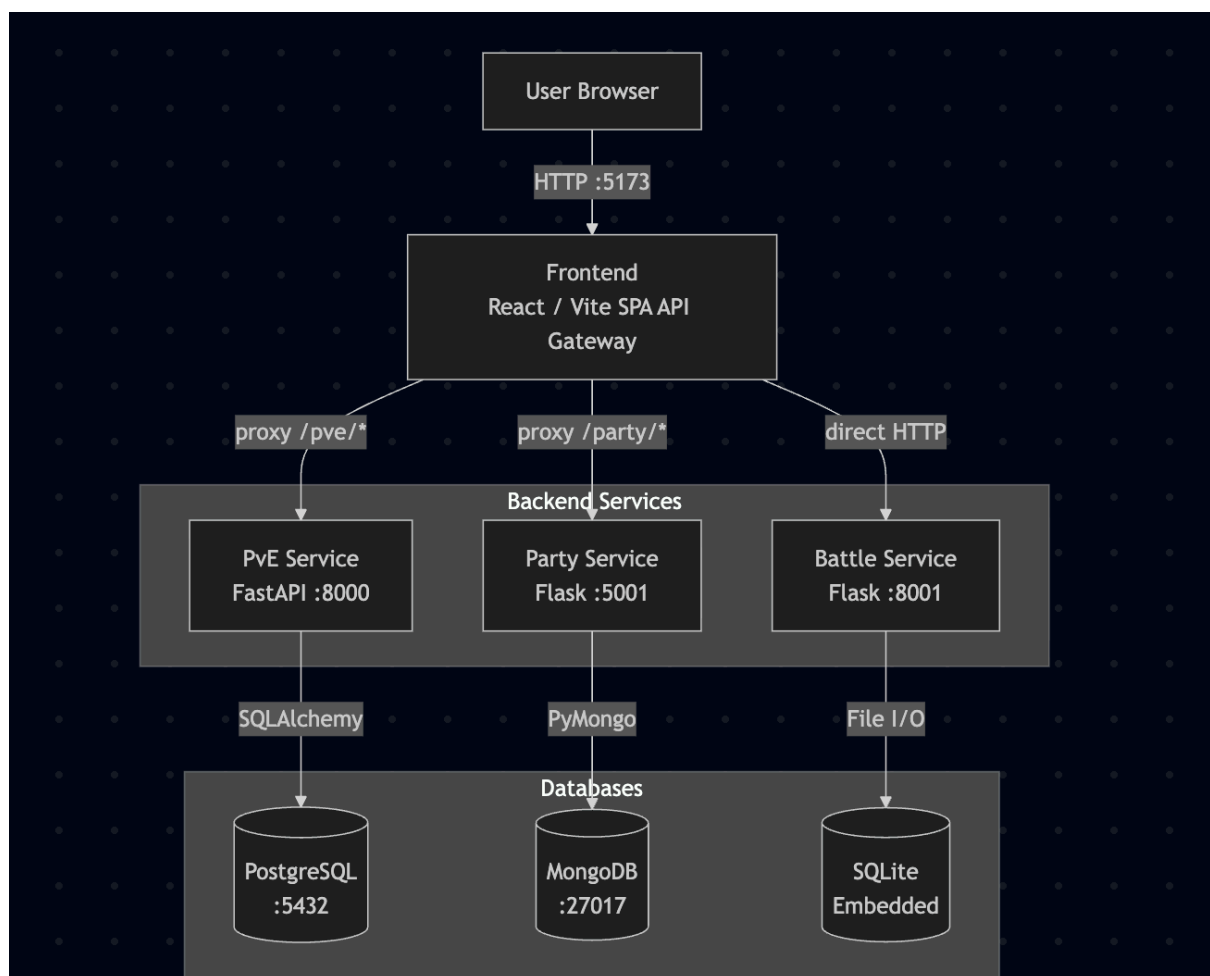
Once the Battle Service declares the fight is over, the frontend takes the final result and manually reports it back to the PvE Service so it can award gold and XP.

## 4. Internal Docker Networking

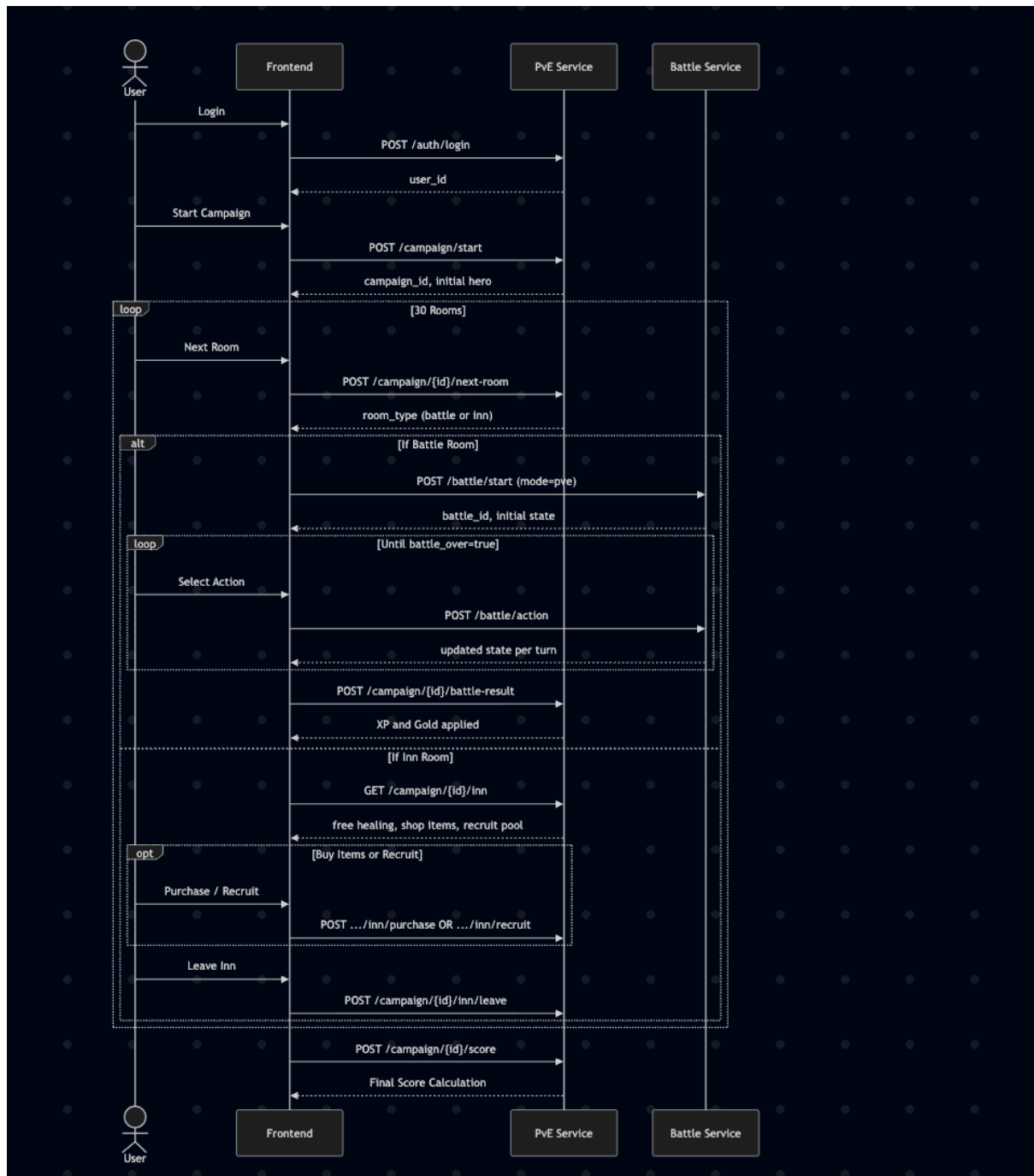
While the frontend drives the main game loops, the backend services are still aware of each other on the internal network. The PvE service is specifically configured with environment variables pointing to the internal Docker URLs of the other services (BATTLE\_SERVICE\_URL: `http://battle-service:8001` and PARTY\_SERVICE\_URL: `http://party-service:5001`).

## Class Diagrams

## Overall Diagram



# PVE SERVICE



## PVP LIFECYCLE

